ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Real-time Weather Data Analysis and Forecasting System in Vietnam

Big Data Storage and Processing– IT4043E
Group 10

ONE LOVE. ONE FUTURE.

HUST

hust.edu.vn fb.com/dhbkhn

Table of contents

I. Problem Definition

- Vietnam's increasing climate variability and the limitations of traditional forecasting methods highlight the need for a real-time, scalable system that integrates high-frequency data to deliver accurate, localized weather predictions.
- The problem is quintessential for "Big Data":

Volume	Historical weather data can reach hundreds of thousands of rows, requiring fast read and write operations.
Velocity	Weather data arrives in real time and must be ingested and processed within seconds to be useful.
Variety	Data comes from multiple sources and formats (JSON and CSV) and must be unified into a single structure.
Veracity	Sensor noise and inconsistent fields require real-time data cleaning and validation
Value	Accurate real-time weather forecasting delivers significant socio-economic benefits for Vietnam.

I. Problem Definition

Scope

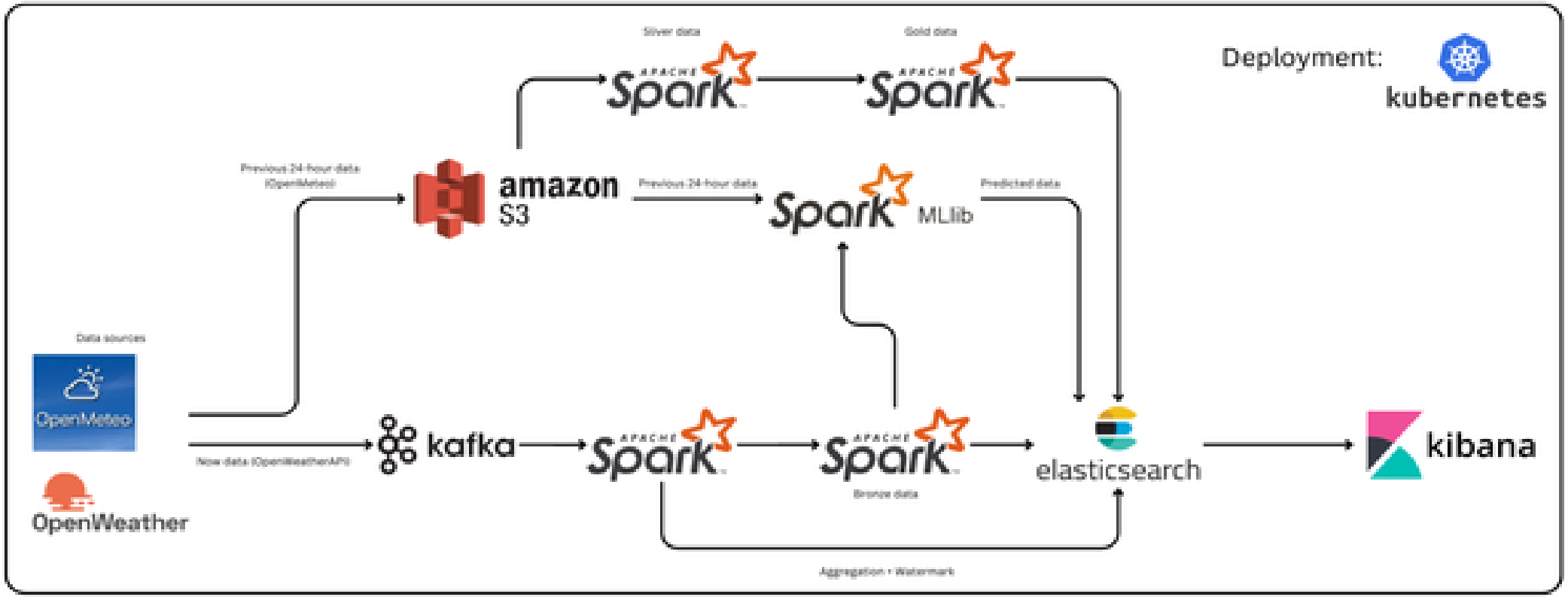
- Geographical Coverage: Analysis of weather data across 34 cities in Vietnam
- Data Sources: Open-source weather data collected via APIs from **OpenWeatherAPI** and **Open-Meteo**
- Temporal Focus: Short-term forecasting using 24-hour input data to predict next-hour temperature (historical data stored on S3 for future extension)
- Output: An interactive dashboard displaying weather analytics and next-hour temperature predictions

Limitations

- Hardware Constraints: High-resolution weather modeling requires substantial CPU/GPU resources and costly cloud infrastructure
- Data Availability: Limited or missing sensor coverage in some regions affects data completeness
- Forecast Uncertainty: Atmospheric systems are inherently chaotic, limiting prediction accuracy over longer time horizons

II. Architecture and Design

- Lambda Architecture covering Ingestion, Processing, Storage, Visualization



II. Architecture and Design

- **Data Ingestion Layer**

OpenMeteo: Historical weather data

OpenWeather Current weather data


```
{
  "latitude": 21.0,
  "longitude": 105.875,
  "generationtime_ms": 0.1646280288696289,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 19.0,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C",
    "relative_humidity_2m": "%",
    "surface_pressure": "hPa",
    "cloud_cover": "%",
    "wind_speed_10m": "km/h",
    "wind_direction_10m": "°",
    "weather_code": "wmo code"
  },
  "hourly": {
    "time": [
      "2025-12-27T00:00", "2025-12-27T01:00", "2025-12-27T02:00",
      ...
    ],
    "temperature_2m": [
      15.0, 15.4, 16.1, ...
    ], ...
```

```
[2025-12-28 13:00:35.980135] Collecting weather data for Hanoi...
Data from OPENWEATHER:
{'base': 'stations',
 'clouds': {'all': 100},
 'cod': 200,
 'coord': {'lat': 21.0245, 'lon': 105.8412},
 'dt': 1766901228,
 'id': 1581130,
 'main': {'feels_like': 19.54,
          'grnd_level': 1017,
          'humidity': 57,
          'pressure': 1018,
          'sea_level': 1018,
          'temp': 20,
          'temp_max': 20,
          'temp_min': 20},
 'name': 'Hanoi',
 'sys': {'country': 'VN',
         'id': 9308,
         'sunrise': 1766878327,
         'sunset': 1766917427,
         'type': 1},
 'timezone': 25200,
 'visibility': 10000,
 'weather': [{'description': 'overcast clouds',
...
```

II. Architecture and Design

- **Data Ingestion Layer**

Approach Tried

- Removed unnecessary columns to reduce data size
- Kept only features needed for ML training and real-time weather display

Final Solution

Dropped unused columns to significantly speed up ML processing

Key Takeaways

- Different APIs have inconsistent data structures and require preprocessing
- Flattening JSON simplifies data for efficient processing
- Reducing data dimensions improves performance in real-time pipelines

[3]:

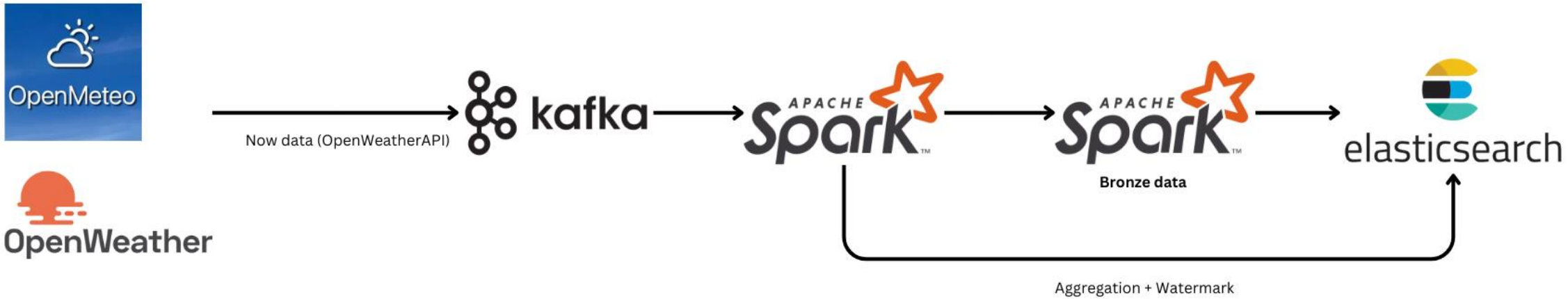
	city	timestamp	description	temp	humidity	pressure	wind_speed	wind_deg	cloudiness	feels_like	temp_min	temp_max	win
0	Hanoi	2025-11-29 07:00:00	clear sky	23.5	50	1011.3	4.00	360	0	NaN	NaN	NaN	
1	Hanoi	2025-11-29 08:00:00	clear sky	23.3	49	1010.4	2.30	39	0	NaN	NaN	NaN	
2	Hanoi	2025-11-29 09:00:00	clear sky	22.7	51	1010.8	2.50	98	0	NaN	NaN	NaN	
3	Hanoi	2025-11-29 10:00:00	clear sky	21.0	62	1010.9	1.80	127	0	NaN	NaN	NaN	
4	Hanoi	2025-11-29 11:00:00	clear sky	19.0	62	1011.6	2.30	162	0	NaN	NaN	NaN	
5	Hanoi	2025-11-29 12:00:00	clear sky	18.3	62	1011.4	2.20	171	0	NaN	NaN	NaN	
6	Hanoi	2025-11-29 13:00:00	clear sky	17.3	65	1011.9	3.20	153	0	NaN	NaN	NaN	
7	Hanoi	2025-11-29 14:00:00	clear sky	16.5	68	1012.0	6.40	133	0	NaN	NaN	NaN	
8	Hanoi	2025-11-29 15:00:00	clear sky	14.7	76	1012.5	4.70	122	0	NaN	NaN	NaN	
9	Hanoi	2025-11-29 16:00:00	clear sky	13.8	79	1012.3	3.90	124	0	NaN	NaN	NaN	

II. Architecture and Design

Stream Processing

Apache Kafka

- Kafka handles stream data. In this case, stream data is the current weather data from OpenWeather API.



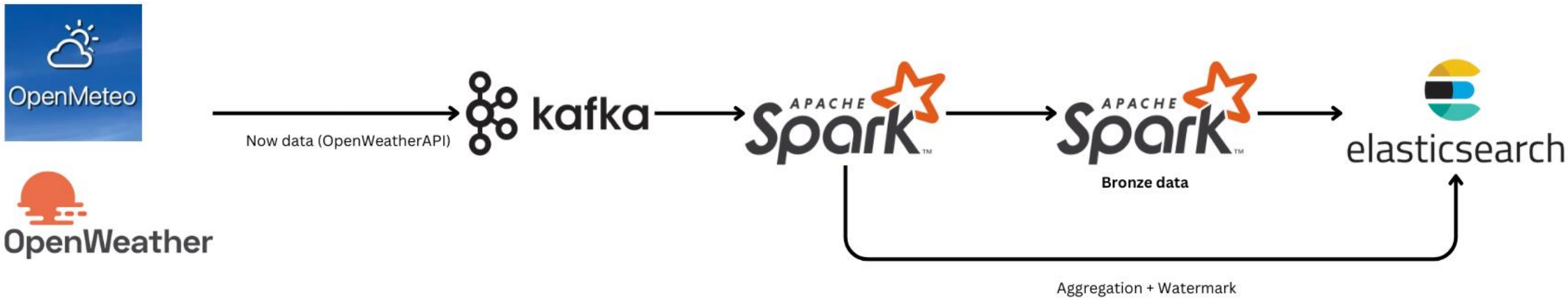

```
{"city": "Gia Lai", "city_id": 1581088, "country": "VN", "event_time": "2025-12-28T05:00:00+00:00", "dt": 1766897730, "sunrise": 1766876925, "sunset": 176691767
2, "timezone": 25200, "coord_lat": 13.75, "coord_lon": 108.25, "weather_id": 804, "weather_main": "Clouds", "weather_description": "overcast clouds", "weather_i
con": "04d", "temp": 25.33, "feels_like": 25.35, "temp_min": 25.33, "temp_max": 25.33, "pressure": 1014, "humidity": 55, "sea_level": 1014, "grnd_level": 963, "
visibility": 10000, "wind_speed": 4.16, "wind_deg": 72, "wind_gust": 5.27, "cloudiness": 91}
{"city": "Hanoi", "city_id": 1581130, "country": "VN", "event_time": "2025-12-28T05:00:00+00:00", "dt": 1766898022, "sunrise": 1766878327, "sunset": 1766917427,
 "timezone": 25200, "coord_lat": 21.0245, "coord_lon": 105.8412, "weather_id": 804, "weather_main": "Clouds", "weather_description": "overcast clouds", "weather
_icon": "04d", "temp": 19, "feels_like": 18.44, "temp_min": 19, "temp_max": 19, "pressure": 1019, "humidity": 57, "sea_level": 1019, "grnd_level": 1018, "visibi
lity": 10000, "wind_speed": 1.36, "wind_deg": 349, "wind_gust": 1.38, "cloudiness": 100}
```

II. Architecture and Design

Stream Processing

Apache Spark (Stream Spark + Bronze)

- Real-time weather data ingested from **OpenWeather API** via **Kafka**
- **Spark Structured Streaming** processes event-time data continuously
- Raw events stored in **Bronze (Parquet)** for durability
- **Windowed aggregation + watermark** handle late data
- Aggregated results **upsurged to Elasticsearch** for real-time analytics

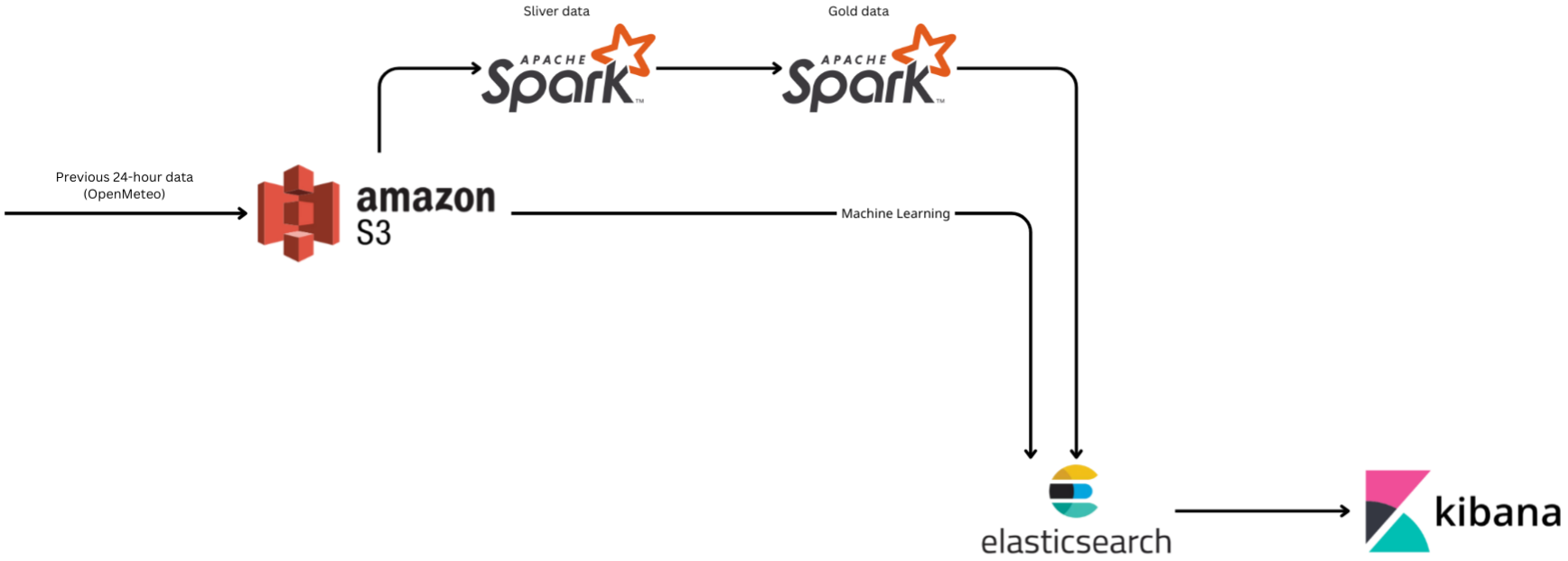


II. Architecture and Design

Batch Processing

Amazon S3

- Amazon S3 serves as a storage for historical data
- Historical weather data stored in **Amazon S3**
- **Spark batch jobs** process 24-hour data windows
- **CronJob runs every 45 minutes** to add datasets
- Processed data written to **Silver layer** and **ML-ready tables**




```
download: s3://hust-buck
city: string
timestamp: timestamp[us]
description: string
temp: double
humidity: double
pressure: double
wind_speed: double
wind_deg: double
cloudiness: double
```

II. Architecture and Design

Batch Processing

Time-Series Data Storage Evaluation

- Large-scale time-series weather data requires efficient and scalable storage.
- Multiple storage formats were evaluated for ingestion and analytics performance

- Row-based storage format.
- Excellent for fast ingestion and schema evolution.
- Inefficient for analytics since entire rows must be read even when querying a single column.

- Columnar format optimized for Hive.
- Highest compression ratio among tested formats.
- Less compatible with Python and broader data science workflows outside Hadoop.

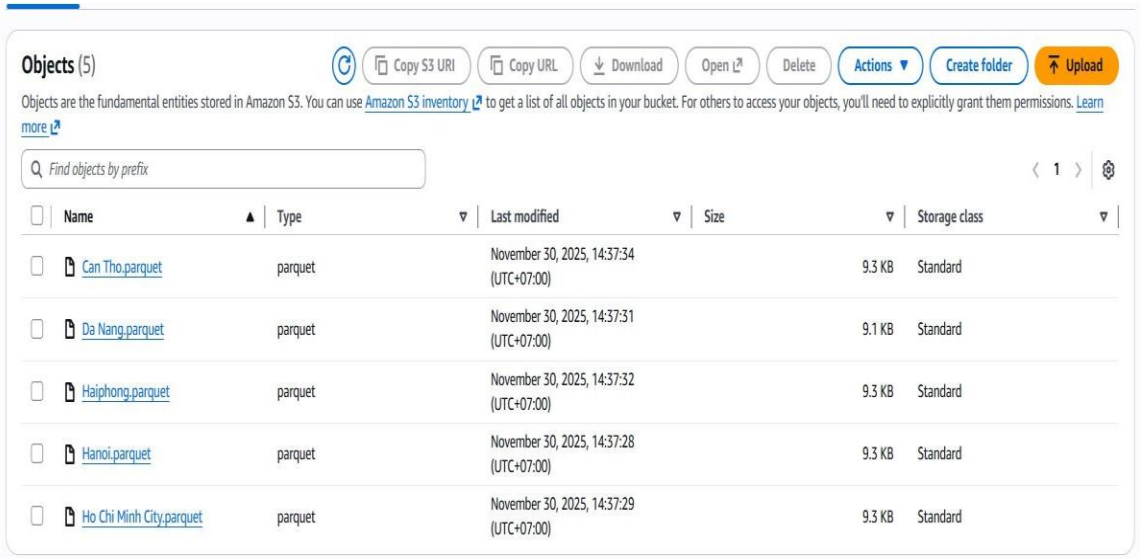
- Columnar format designed for analytical workloads and nested data.
- Well-suited for transformed datasets in Silver and Gold layers.
- Strong ecosystem support across Spark and Python tools.

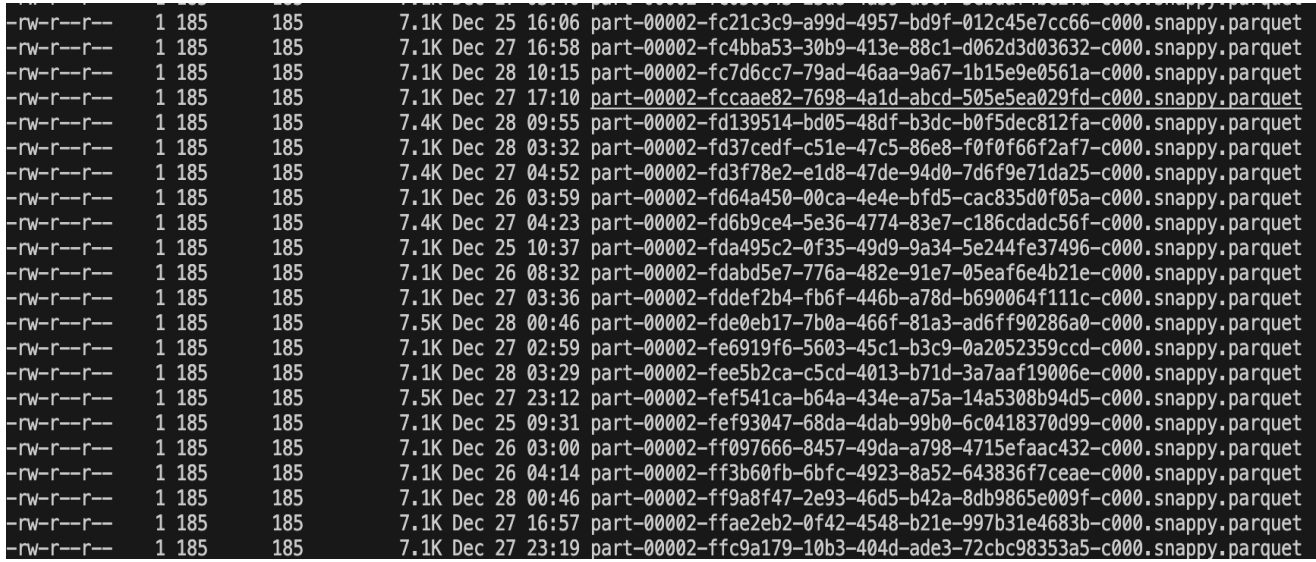
II. Architecture and Design

Batch Processing

Parquet chosen as the primary storage format

- Best balance of performance, usability, and ecosystem compatibility.
- Easier to work with than Avro and ORC for analytics and ML pipelines.



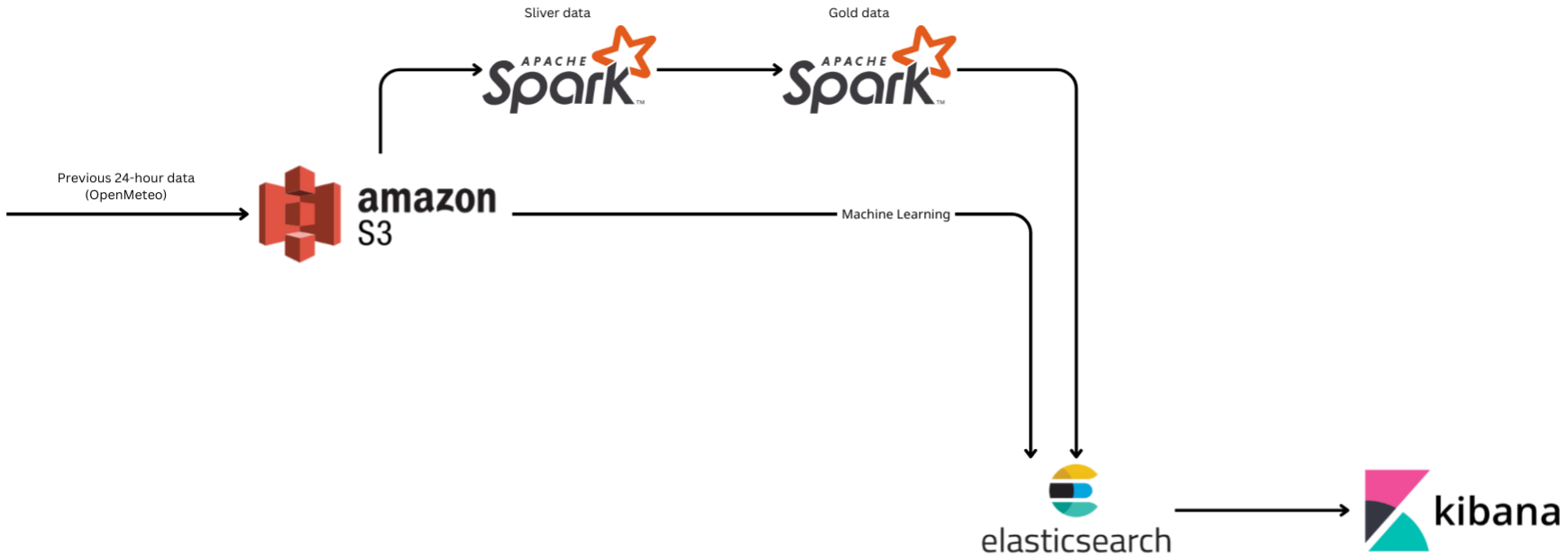


II. Architecture and Design

Batch Processing

Silver data

- **Read historical data** stored in Amazon S3
- **Spark batch job runs hourly (CronJob)**
- Cleans, deduplicates, and add average tempurature in 24 hours
- Outputs **Silver data** for **Gold data**

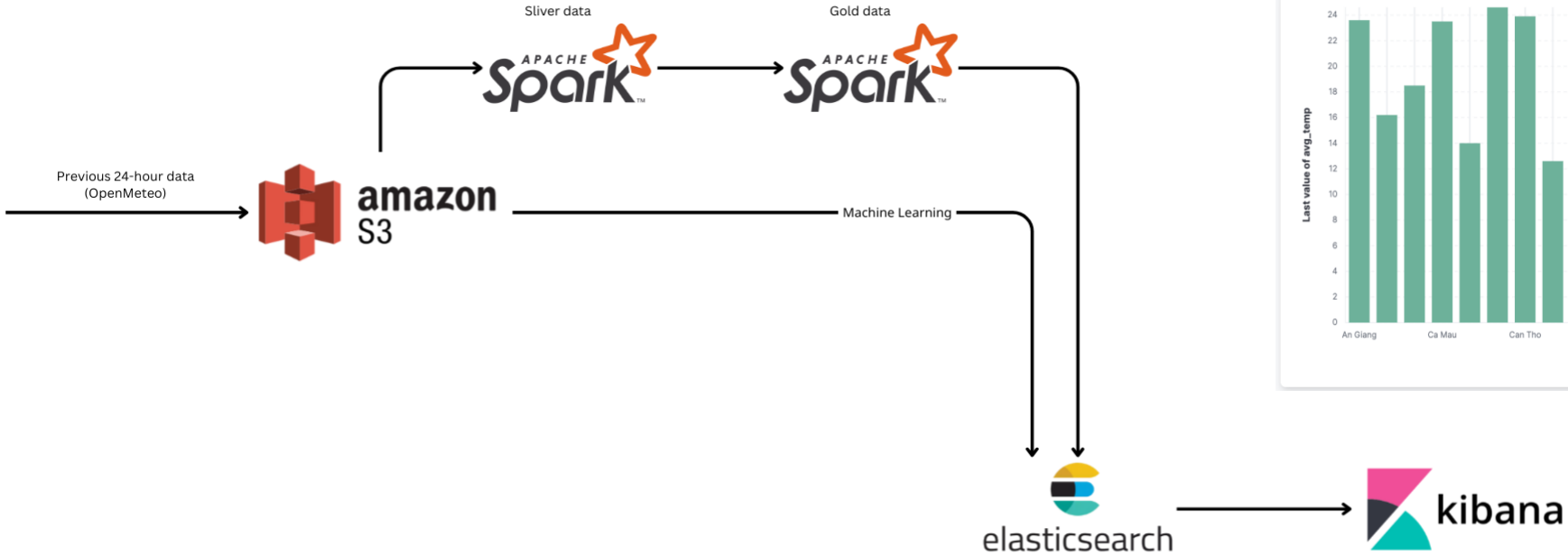


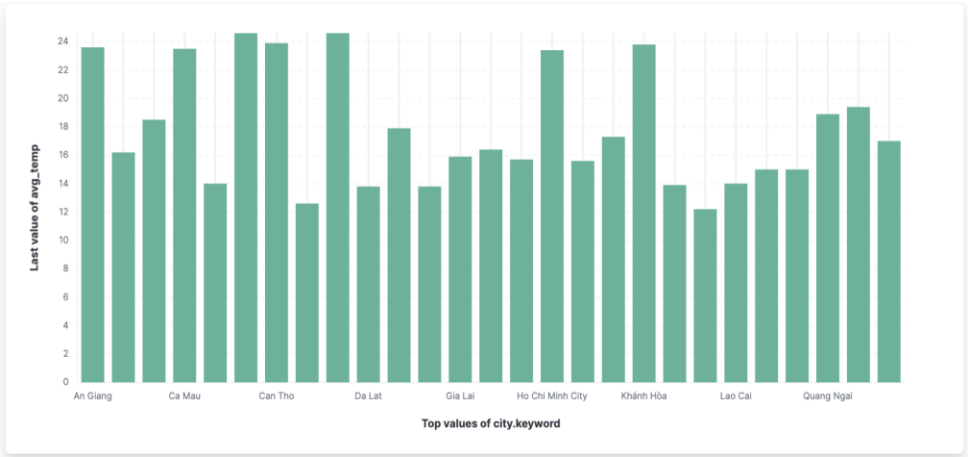
II. Architecture and Design

Batch Processing

Gold data

- **Spark batch job runs every 1h10m (CronJob)**
- Aggregates **Silver data → hourly city-level metrics**
- Computes alerts & derived features
- Writes **Gold data to Elasticsearch for dashboards**



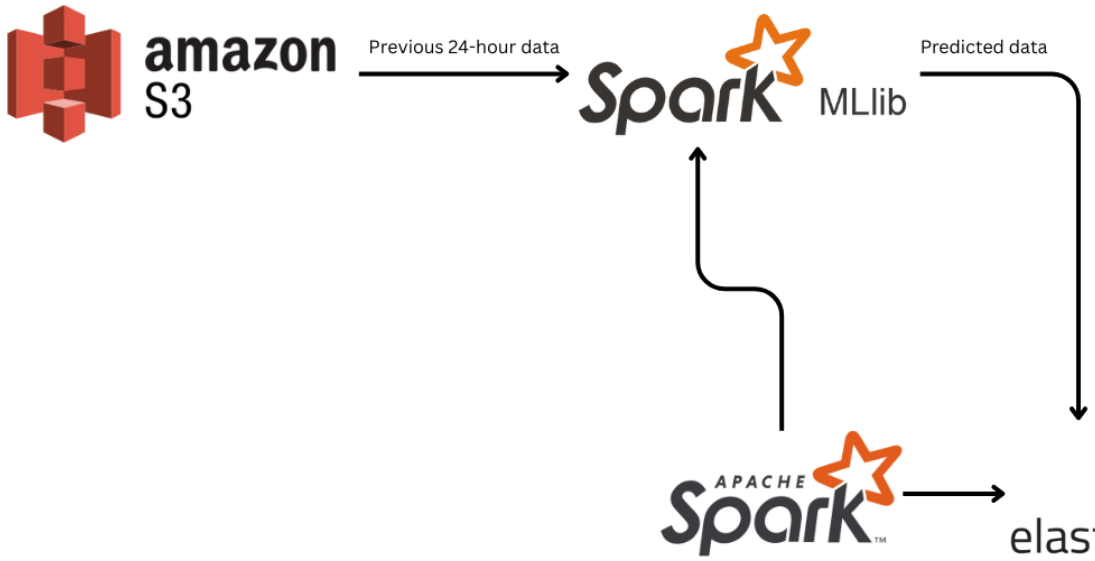


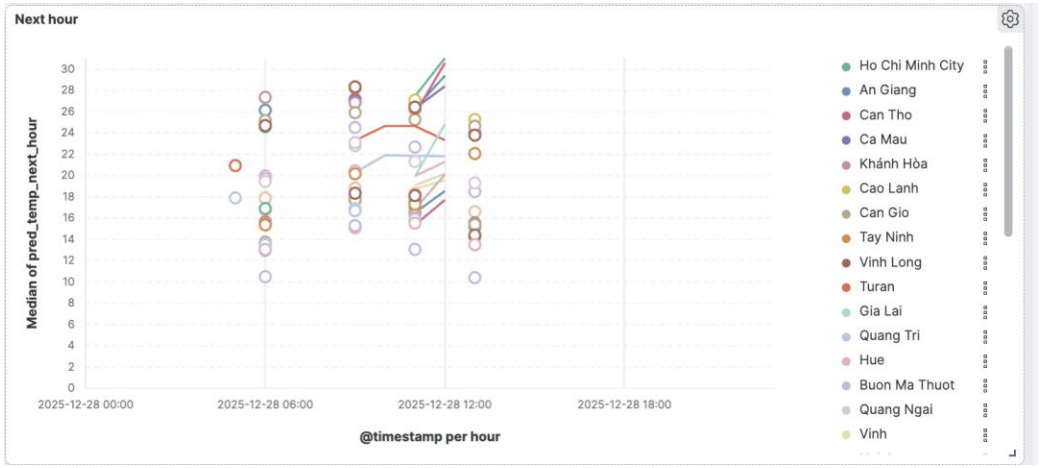
II. Architecture and Design

Machine Learning model Implementation

Spark MLlib

- **Hourly batch inference with Spark MLlib**
- Uses **last 24h S3 + recent Bronze data**
- Predicts **next-hour temperature per city**
- Stores predictions in **Elasticsearch for visualization**



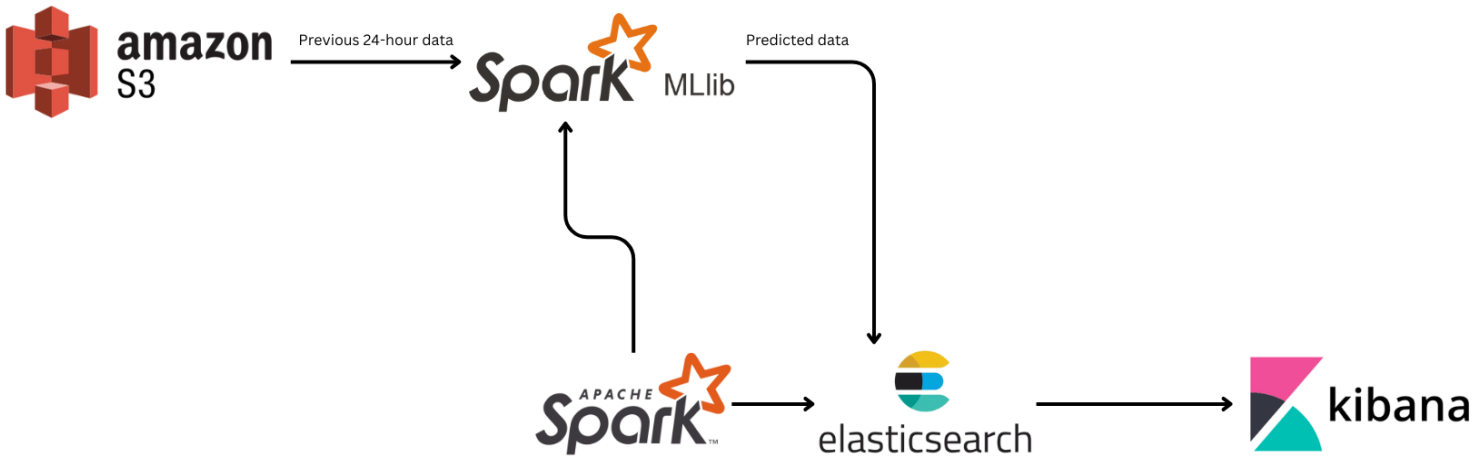


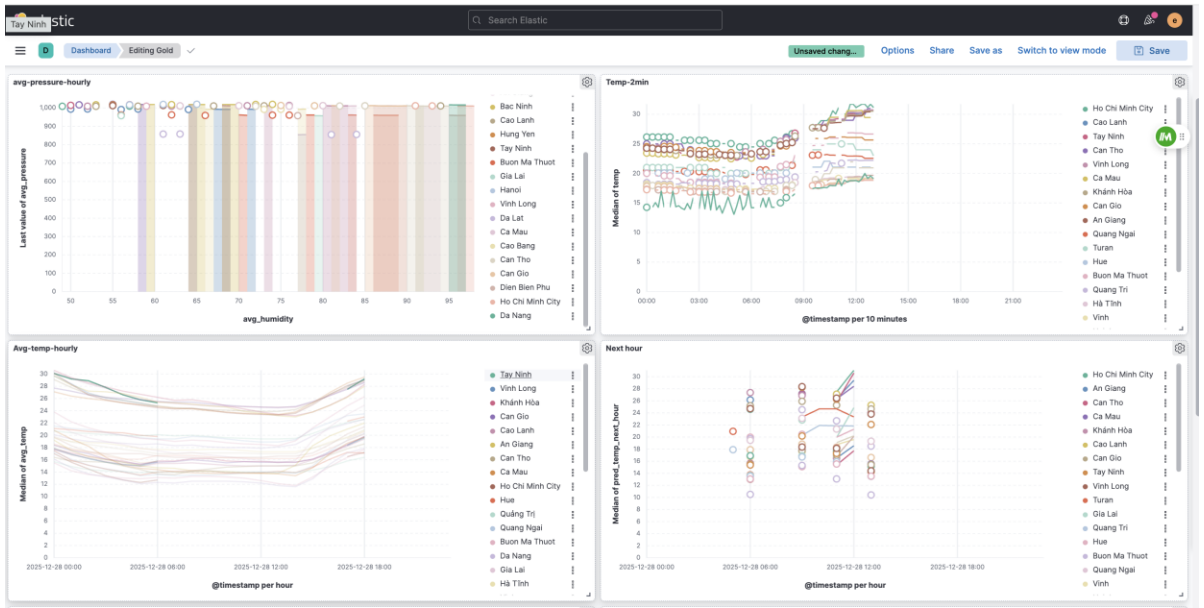
II. Architecture and Design

Visualization

Kibana

- Monitoring & analytics
- Gold + ML outputs
- Interactive dashboards



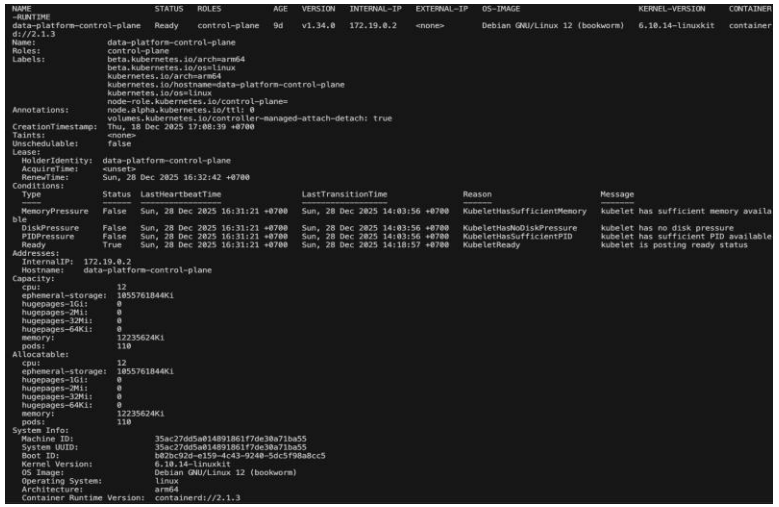


II. Architecture and Design

Deployment

Kubernetes

- Kubernetes provides a **unified orchestration layer** to run streaming, batch, and ML workloads on the same infrastructure.
- It handles **resource scheduling and isolation**, allowing Spark, Kafka, and Elasticsearch to coexist without hard coupling.
- Kubernetes enables **fault recovery and restart** of long-running services such as Kafka brokers and Spark streaming jobs.
- It simplifies **deployment and lifecycle management** through declarative manifests and operators (Strimzi, Spark Operator, Elastic Operator).



```
NAME                    STATUS   ROLES          AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                       KERNEL-VERSION    CONTAINER
-RUNTIME
data-platform-control-plane   Ready    control-plane   9d    v1.34.0   172.19.0.2    <none>        Debian GNU/Linux 12 (bookworm)   6.10.14-linuxkit   container
d://2.1.3
Name:               data-platform-control-plane
Roles:              control-plane
Labels:             beta.kubernetes.io/arch=arm64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=arm64
                    kubernetes.io/hostname=data-platform-control-plane
                    kubernetes.io/os=linux
                    node-role.kubernetes.io/control-plane=
Annotations:        node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Thu, 18 Dec 2025 17:08:39 +0700
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  data-platform-control-plane
  AcquireTime:     <unset>
  RenewTime:       Sun, 28 Dec 2025 16:32:42 +0700
Conditions:
  Type
  ----
  MemoryPressure   False   Sun, 28 Dec 2025 16:31:21 +0700   Sun, 28 Dec 2025 14:03:56 +0700   KubeletHasSufficientMemory   kubelet has sufficient memory availa
ble
  DiskPressure     False   Sun, 28 Dec 2025 16:31:21 +0700   Sun, 28 Dec 2025 14:03:56 +0700   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Sun, 28 Dec 2025 16:31:21 +0700   Sun, 28 Dec 2025 14:03:56 +0700   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Sun, 28 Dec 2025 16:31:21 +0700   Sun, 28 Dec 2025 14:10:57 +0700   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  172.19.0.2
  Hostname:    data-platform-control-plane
Capacity:
  cpu:                12
  ephemeral-storage:  1055761844Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  hugepages-32Mi:     0
  hugepages-64Ki:     0
  memory:             12235624Ki
  pods:               110
Allocatable:
  cpu:                12
  ephemeral-storage:  1055761844Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  hugepages-32Mi:     0
  hugepages-64Ki:     0
  memory:             12235624Ki
  pods:               110
System Info:
  Machine ID:                 35ac27dd5a0148918617de30a71ba55
  System UUID:                35ac27dd5a0148918617de30a71ba55
  Boot ID:                    b02bc920-e159-4c43-9240-5dc5f98a8cc5
  Kernel Version:             6.10.14-linuxkit
  OS Image:                   Debian GNU/Linux 12 (bookworm)
  Operating System:           linux
  Architecture:               arm64
  Container Runtime Version:  containerd://2.1.3
```

III. Implementation Details

Kafka Producer (Data sources readers): *Weather-producer.yaml* (for kafka)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: weather-kafka-producer
  namespace: data
spec:
  replicas: 1
  selector:
    matchLabels:
      app: weather-kafka-producer
  template:
    metadata:
      labels:
        app: weather-kafka-producer
    spec:
      containers:
        - name: producer
          image: weather-kafka-producer:latest
          imagePullPolicy: IfNotPresent
          env:
            - name: KAFKA_BROKER
              value: kafka-kafka-bootstrap.data.svc.cluster.local:9092
            - name: KAFKA_TOPIC
              value: weather-raw
            - name: OPENWEATHER_API_KEY
              valueFrom:
                secretKeyRef:
                  name: openweather-secret
                  key: api-key
            - name: POLL_INTERVAL
              value: "60"
```

Lifecycle

1.Deployment creates Pod
2.Pod starts Python script
3.Script:
    1. Fetches weather
    2. Produces messages to Kafka
4.If script crashes:
    1. Pod restarts
    2. Producer reconnects automatically

III. Implementation Details

S3

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-history-to-s3
  namespace: data
spec:
  schedule: "*/45 * * * *" # mỗi 45 phút
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 2

  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure

          containers:
            - name: s3-history
              image: weather-s3-history:latest
              imagePullPolicy: IfNotPresent

              env:
                - name: S3_BUCKET_NAME
                  value: hust-bucket-storage

                - name: AWS_ACCESS_KEY_ID
                  valueFrom:
                    secretKeyRef:
                      name: aws-secret
                      key: access-key
```

III. Implementation Details

Bronze

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: bronze-to-es-batch-cron
  namespace: default
spec:
  schedule: "*/2 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  cat <<'EOF' | kubectl apply -f -
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

Silver.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-s3-to-silver-cron
  namespace: default
spec:
  schedule: "0 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-silver-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

III. Implementation Details

Gold

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: silver-to-gold-es-cron
  namespace: default
spec:
  schedule: "10 * * * *" # chạy sau silver 10 phút
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-gold-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

Machine Learning Implementation

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: weather-ml-inference-cron
  namespace: default
spec:
  schedule: "0 * * * *" # ⏱ mỗi giờ
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3

  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: spark
          restartPolicy: Never

          containers:
            - name: submit-ml-spark-job
              image: bitnami/kubectl:latest
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - |
                  kubectl create -f - <<'EOF'
                  apiVersion: sparkoperator.k8s.io/v1beta2
                  kind: SparkApplication
                  metadata:
```

III. Implementation Details

Elasticsearch

```yaml
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: es-weather
  namespace: observability
spec:
  version: 7.17.9
  http:
    tls:
      selfSignedCertificate:
        disabled: true
  nodeSets:
    - name: default
      count: 1
      config:
        node.store.allow_mmap: false
      podTemplate:
        spec:
          containers:
            - name: elasticsearch
              resources:
                requests:
                  memory: 1Gi
                  cpu: 250m
                limits:
                  memory: 1Gi
                  cpu: 1
```

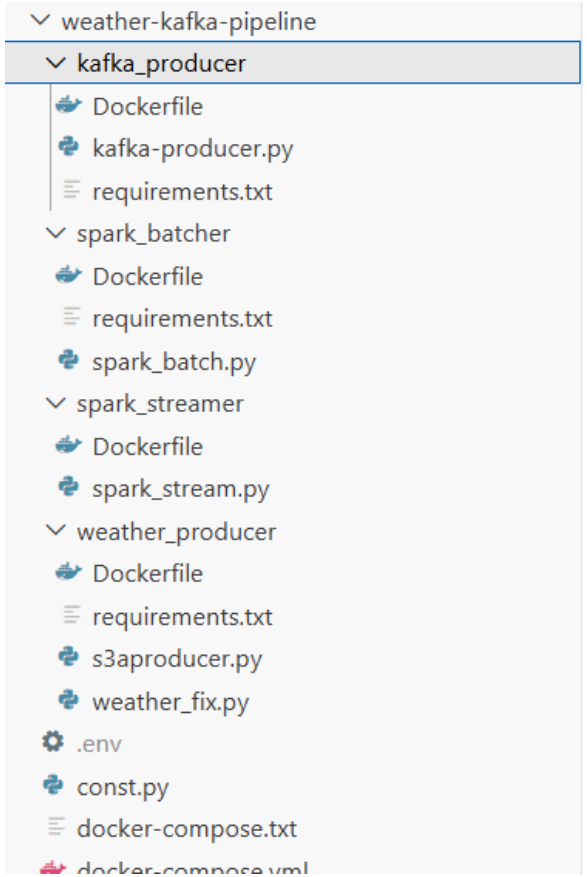
Kibana

```yaml
apiVersion: kibana.k8s.elastic.co/v1
kind: Kibana
metadata:
  name: kibana-weather
  namespace: observability
spec:
  version: 7.17.9
  count: 1

  elasticsearchRef:
    name: es-weather

  http:
    tls:
      selfSignedCertificate:
        disabled: true # dễ port-forward, debug

  podTemplate:
    spec:
      containers:
        - name: kibana
          resources:
            requests:
              cpu: 200m
              memory: 512Mi
            limits:
              cpu: 1
              memory: 1Gi
```

III. Implementation Details

Deployment strategy

We first deploy on docker using many Dockerfiles, with a docker-compose.yml file to run everything




```yaml
spark-streamer:
  build:
    context: ./spark_streamer
  # volumes:
  #   - ./spark_stream.py:/app/spark_stream.py
  container_name: spark-streamer
  depends_on:
    - kafka
    - elasticsearch
  networks:
    - weather-net
```

▷Run Service

```yaml
spark-batcher:
  build:
    context: ./spark_batcher      # Folder containing Dockerfile + script
  container_name: spark-batcher
  depends_on:
    - kafka
    - elasticsearch
  env_file:
    - .env
  networks:
    - weather-net
```

III. Implementation Details

Deployment strategy

Kubernetes

- Deployed on a **single-node Kubernetes cluster**, where one node acts as both **control plane and worker**.
- Uses **containerd** as the container runtime with CPU and memory sized for data processing workloads.
- Employs **PersistentVolumeClaims (PVCs)** for Spark checkpoints and intermediate data storage.
- Organizes services into **separate namespaces** to ensure logical isolation and easier management.

```
PodCIDR:                      10.244.0.0/24
PodCIDRs:                     10.244.0.0/24
ProviderID:                   kind://docker/data-platform/data-platform-control-plane
Non-terminated Pods:          (29 in total)
  Namespace                   Name                                                    CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                                                    ------------  ----------  ---------------  -------------  ---
  data                        kafka-entity-operator-5968477844-cnhzk                  0 (0%)        0 (0%)      0 (0%)           0 (0%)         17h
  data                        kafka-kafka-pool-0                                      0 (0%)        0 (0%)      0 (0%)           0 (0%)         109m
  data                        kafka-kafka-pool-1                                      0 (0%)        0 (0%)      0 (0%)           0 (0%)         113m
  data                        kafka-kafka-pool-2                                      0 (0%)        0 (0%)      0 (0%)           0 (0%)         113m
  data                        strimzi-cluster-operator-8457d7566-hx2cn                200m (1%)     1 (8%)      384Mi (3%)       384Mi (3%)     9d
  data                        weather-kafka-producer-5d7b968f59-f5c6p                 0 (0%)        0 (0%)      0 (0%)           0 (0%)         3d
  default                     bronze-to-es-batch-97b6f89b644cfc7c-exec-1              1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   41s
  default                     bronze-to-es-batch-97b6f89b644cfc7c-exec-2              1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   41s
  default                     bronze-to-es-batch-driver                               1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   44s
  default                     checkpoint-debug                                        0 (0%)        0 (0%)      0 (0%)           0 (0%)         8d
  default                     silver-debug                                            0 (0%)        0 (0%)      0 (0%)           0 (0%)         8d
  default                     spark-debug                                             0 (0%)        0 (0%)      0 (0%)           0 (0%)         16h
  default                     weather-streaming-to-bronze-b311d79b638cfba4-exec-1      1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   3h30m
  default                     weather-streaming-to-bronze-b311d79b638cfba4-exec-2      1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   3h30m
  default                     weather-streaming-to-bronze-driver                      1 (8%)        0 (0%)      1433Mi (11%)     1433Mi (11%)   3h30m
  elastic-system              elastic-operator-0                                      100m (0%)     1 (8%)      150Mi (1%)       1Gi (8%)       9d
  kube-system                 coredns-66bc5c9577-4nllb                                100m (0%)     0 (0%)      70Mi (0%)        170Mi (1%)     9d
  kube-system                 coredns-66bc5c9577-lx5d5                                100m (0%)     0 (0%)      70Mi (0%)        170Mi (1%)     9d
  kube-system                 etcd-data-platform-control-plane                        100m (0%)     0 (0%)      100Mi (0%)       0 (0%)         9d
  kube-system                 kindnet-hj7ln                                           100m (0%)     100m (0%)   50Mi (0%)        50Mi (0%)      9d
  kube-system                 kube-apiserver-data-platform-control-plane              250m (2%)     0 (0%)      0 (0%)           0 (0%)         9d
  kube-system                 kube-controller-manager-data-platform-control-plane     200m (1%)     0 (0%)      0 (0%)           0 (0%)         9d
  kube-system                 kube-proxy-zcgtd                                        0 (0%)        0 (0%)      0 (0%)           0 (0%)         9d
  kube-system                 kube-scheduler-data-platform-control-plane              100m (0%)     0 (0%)      0 (0%)           0 (0%)         9d
  local-path-storage          local-path-provisioner-7b8c8ddbd6-54m6x                 0 (0%)        0 (0%)      0 (0%)           0 (0%)         9d
  observability               es-weather-es-default-0                                 250m (2%)     1 (8%)      1Gi (8%)         1Gi (8%)       9d
  observability               kibana-weather-kb-76fd7567b8-f4xv6                      200m (1%)     1 (8%)      512Mi (4%)       1Gi (8%)       8d
  spark-operator              spark-operator-controller-5fbfcb9d47-wshds              0 (0%)        0 (0%)      0 (0%)           0 (0%)         9d
  spark-operator              spark-operator-webhook-55c6f74b59-xn4vm                 0 (0%)        0 (0%)      0 (0%)           0 (0%)         9d
```

```
(big-data) minh@MacBook-Pro-cua-Minh kubernetes % kubectl get ns -o name
namespace/data
namespace/default
namespace/elastic-system
namespace/kafka
namespace/kube-node-lease
namespace/kube-public
namespace/kube-system
namespace/local-path-storage
namespace/observability
namespace/spark-operator
```

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests      Limits
  --------           --------      ------
  cpu                7700m (64%)   4100m (34%)
  memory             10958Mi (91%)  12444Mi (104%)
  ephemeral-storage  0 (0%)        0 (0%)
  hugepages-1Gi      0 (0%)        0 (0%)
  hugepages-2Mi      0 (0%)        0 (0%)
  hugepages-32Mi     0 (0%)        0 (0%)
  hugepages-64Ki     0 (0%)        0 (0%)
Events:              <none>
```

IV. Lesson learned

System Integration

- Enable data flow from **Kafka to Spark** and from **Spark to Elasticsearch**.
- Kafka and Spark run on **Kubernetes**, requiring proper service configuration for communication.

Approaches Tried

- **Docker Compose Configuration**
  - Used docker-compose.yml to connect Kafka, Spark, and Elasticsearch.
  - Effective for local development and testing.
- **Kubernetes YAML Configuration**
  - Used .yaml manifests to define services, deployments, and networking.
  - Kubernetes-native approach for inter-service communication.

Final Solution: Kubernetes YAML files selected

- Required for Kubernetes-based deployment.
- Provides full control over networking, scaling, and service discovery.

IV. Lesson learned

Performance Optimization

- **Problem**: Frequent loading of S3 data for each ML inference.
- **Approach**:
  - Cache historical data in Spark DataFrames.
  - Load only the previous 24 hours at the start of each hour.
- **Final Solution**: Use cached data instead of reloading from S3.

Monitoring & Debugging

Problem: Need system logs and visibility for monitoring.

Approaches

- Kubernetes CLI commands (kubectl describe, logs).
- Grafana-based monitoring.

Final Solution

- Kubernetes-native commands used.
- Grafana rejected due to high resource overhead.

IV. Lesson learned

Scaling

- **Problem:** Improve Spark processing performance.
- **Approach:**
  - Increased Spark executors from 1 to 2.
  - Enabled parallel processing of data partitions.
- **Final Solution:** Two Spark executors used in production.

Data Quality & Testing

- **Problem:** Prevent broken pipelines before pushing code to GitHub.
- **Approaches**
  - Local unit testing of individual components.
  - Manual testing of data flow.
  - Docker used to simulate production services.
- **Final Solution:** Combine local testing with container-based integration testing.

IV. Lesson learned

Data Quality & Testing

- **Problem**
  - Prevent broken pipelines before pushing code to GitHub.
- **Approaches**
  - Local unit testing of individual components.
  - Manual testing of data flow.
  - Docker used to simulate production services.
- **Final Solution**
  - Combine local testing with container-based integration testing.
- **Key Takeaways**
  - Early testing reduces deployment errors.
  - Component isolation simplifies debugging.

V. Conclusion

In conclusion, we have ...

- This project successfully builds an end-to-end real-time weather data pipeline for Vietnam using modern Big Data technologies.
- The system integrates Kafka, Spark (streaming & batch), Elasticsearch, and Kubernetes to process, store, and analyze high-velocity weather data.
- A Lambda Architecture is applied to support both real-time analytics and historical batch processing.
- The pipeline demonstrates scalability, fault tolerance, and modular deployment suitable for real-world data engineering systems.

Future works

- Extend forecasting models with advanced ML/DL approaches (e.g., LSTM, Transformer-based time-series models).
- Expand data sources to include additional meteorological APIs and sensor networks.
- Deploy the system on a multi-node Kubernetes cluster to evaluate scalability and high availability.
- Optimize resource usage and monitoring with lightweight observability solutions.